
Document d'Installation et de Configuration

Plateforme Data ELT

Projet	Automatisation et modernisation de la plateforme data
Architecture	NiFi → BigQuery (staging) → dbt → BigQuery (dwh)
Orchestration	Apache Airflow (Docker Compose)
Réalisé par	Amine Machal
Encadré par	Mr. Elyusufi
Date	July 13, 2026

Stack technique utilisée

Extract & Load

Apache NiFi

Transform

dbt + BigQuery

Orchestration

Apache Airflow

Contents

1	Introduction	3
1.1	Architecture générale	3
1.2	Prérequis système	3
2	Installation de PostgreSQL	3
2.1	Téléchargement et installation	3
2.2	Création de la base de données	4
3	Installation d'Apache NiFi	4
3.1	Téléchargement	4
3.2	Démarrage de NiFi	4
3.3	Accès à l'interface	4
3.4	Installation du driver JDBC PostgreSQL	4
3.5	Configuration du Controller Service DBCPConnectionPool	5
4	Configuration de Google BigQuery	5
4.1	Création du projet Google Cloud	5
4.2	Activation de l'API BigQuery	5
4.3	Création des datasets	5
4.4	Création du compte de service	6
5	Configuration de NiFi pour BigQuery	6
5.1	Controller Service GCPCredentialsControllerService	6
5.2	Flux d'ingestion vers BigQuery	6
5.2.1	Configuration QueryDatabaseTable	6
5.2.2	Configuration PutBigQuery	7
6	Installation et configuration de dbt	7
6.1	Installation	7
6.2	Initialisation du projet	7
6.3	Configuration du fichier profiles.yml	7
6.3.1	Pour les tests locaux (Windows)	7
6.3.2	Pour Airflow (conteneur Docker)	8
6.4	Structure des modèles dbt	8
6.5	Test de la connexion	8
7	Installation d'Apache Airflow (Docker Compose)	9
7.1	Prérequis	9
7.2	Téléchargement du fichier docker-compose.yaml	9
7.3	Modifications du docker-compose.yaml	9
7.3.1	1. Installer dbt-bigquery au démarrage	9
7.3.2	2. Monter le projet dbt en volume	9
7.3.3	3. Ajouter le nom de la machine pour NiFi	10
7.4	Démarrage d'Airflow	10
7.5	Accès à l'interface	10
7.6	Placement du DAG	10

8	Configuration du DAG Airflow	10
8.1	Variables à configurer dans le DAG	11
8.2	Planification	11
9	Vérification de l'installation complète	11
9.1	Checklist de vérification	11
9.2	Test end-to-end	11

1 Introduction

Ce document décrit les étapes d'installation et de configuration de l'environnement technique nécessaire pour faire fonctionner le pipeline ELT (Extract, Load, Transform) de la plateforme data.

1.1 Architecture générale

Le pipeline suit le flux suivant :

PostgreSQL (source) *NiFi* BigQuery (staging) *dbt* BigQuery (dwh)
Orchestration globale par Apache Airflow

1.2 Prérequis système

Composant	Version minimale	Rôle
Windows	10 / 11	Système d'exploitation hôte
Docker Desktop	4.x	Conteneurisation d'Airflow
Python	3.8+	Environnement dbt (machine locale)
Java	11+	Requis par Apache NiFi
PostgreSQL	13+	Base de données source

Table 1: Prérequis système

2 Installation de PostgreSQL

2.1 Téléchargement et installation

1. Télécharger l'installateur officiel depuis <https://www.postgresql.org/download/windows/>
2. Lancer l'installateur et garder les composants par défaut cochés :
 - PostgreSQL Server
 - pgAdmin 4
 - Command Line Tools
3. Port par défaut : **5432** (laisser tel quel)
4. Définir un mot de passe pour l'utilisateur **postgres** (à noter précieusement)

2.2 Création de la base de données

Ouvrir pgAdmin 4, se connecter au serveur, puis créer la base :

```
1 -- Dans le Query Tool de pgAdmin
2 CREATE DATABASE mabase;
```

Listing 1: Création de la base de données

3 Installation d'Apache NiFi

3.1 Téléchargement

1. Aller sur <https://nifi.apache.org/download.html>
2. Télécharger la version **2.10.x** (Binary, .zip)
3. Extraire l'archive dans C:\nifi-2.10.0-bin\

3.2 Démarrage de NiFi

Ouvrir PowerShell en tant qu'administrateur :

```
1 cd C:\nifi-2.10.0-bin\nifi-2.10.0\bin
2 .\run-nifi.bat
```

Listing 2: Démarrage de NiFi

[i] Information

Au premier démarrage, NiFi génère automatiquement un nom d'utilisateur et un mot de passe dans les logs. Récupérez-les dans le fichier :

C:\nifi-2.10.0-bin\nifi-2.10.0\logs\nifi-app.log

Cherchez les lignes : **Generated Username** et **Generated Password**

3.3 Accès à l'interface

Ouvrir le navigateur sur : <https://localhost:8443/nifi>

3.4 Installation du driver JDBC PostgreSQL

1. Télécharger le driver depuis <https://jdbc.postgresql.org/download/>
2. Prendre le fichier **postgresql-42.x.x.jar**
3. Créer le dossier C:\nifi-2.10.0-bin\nifi-2.10.0\credentials\
4. Placer le fichier **.jar** dans ce dossier

3.5 Configuration du Controller Service DBCPConnectionPool

Dans l'interface NiFi :

1. Clic droit sur le canvas → **Configure** → onglet **Controller Services**
2. Ajouter **DBCPConnectionPool**
3. Configurer les propriétés :

Propriété	Valeur
Database Connection URL	jdbc:postgresql://localhost:5432/mabase
Database Driver Class Name	org.postgresql.Driver
Database Driver Location(s)	C:\nifi-2.10.0-bin\...\postgresql-42.x.x.jar
Database User	postgres
Password	<i>votre mot de passe</i>

Table 2: Configuration DBCPConnectionPool

4 Configuration de Google BigQuery

4.1 Création du projet Google Cloud

1. Aller sur <https://console.cloud.google.com>
2. Créer un nouveau projet (ex. pipeline-elt-test)
3. Noter l'**ID du projet** (visible sur la page d'accueil)

4.2 Activation de l'API BigQuery

Dans la barre de recherche de la console : taper **BigQuery API** → Activer.

4.3 Création des datasets

Dans la console BigQuery (<https://console.cloud.google.com/bigquery>) :

1. Clic sur le projet → **Créer un ensemble de données**
2. Créer **staging** (région : europe-west1)
3. Créer **dwh** (même région)

Les deux datasets doivent être dans la **même région** (ex. europe-west1) pour que dbt puisse faire des jointures entre eux.

4.4 Création du compte de service

1. Console Google Cloud → **IAM et administration** → **Comptes de service**
2. **Créer un compte de service** : nom pipeline-service-account
3. Rôle : **BigQuery Admin**
4. Onglet **Clés** → **Ajouter une clé** → format **JSON**
5. Sauvegarder le fichier bigquery-key.json dans :
 - C:\nifi-2.10.0-bin\nifi-2.10.0\credentials\bigquery-key.json
 - C:\Users\AMINE\mon_projet_stg_sofac\bigquery-key.json

Ne jamais partager ce fichier JSON publiquement ni le mettre sur GitHub, car il donne un accès complet au projet BigQuery.

5 Configuration de NiFi pour BigQuery

5.1 Controller Service GPCredentialsControllerService

1. Dans NiFi → **Controller Services** → ajouter **GPCredentialsControllerService**
2. Configurer :
 - **Authentication Strategy** : Service Account Credentials (Json File)
 - **Service Account JSON File** : chemin vers bigquery-key.json
3. Activer le service (icône)

5.2 Flux d'ingestion vers BigQuery

Pour chaque table source (clients et commandes), créer un flux composé de :

QueryDatabaseTable → PutBigQuery

5.2.1 Configuration QueryDatabaseTable

Propriété	Valeur
Database Connection Pooling Service	DBCPCConnectionPool
Database Type	PostgreSQL
Table Name	source_system.clients (ou commandes)
Maximum-value Columns	client_id (ou commande_id)

Table 3: Configuration QueryDatabaseTable

Propriété	Valeur
GCP Credentials Provider Service	GCPCredentialsControllerService
Project ID	pipeline-elt-test
Dataset	staging
Table Name	stg_clients_raw (ou stg_commandes_raw)
Record Reader	AvroReader

Table 4: Configuration PutBigQuery

5.2.2 Configuration PutBigQuery

6 Installation et configuration de dbt

6.1 Installation

Ouvrir PowerShell :

```
1 pip install dbt-bigquery
```

Listing 3: Installation de dbt

6.2 Initialisation du projet

```
1 dbt init mon_projet_stg_sofac
```

Listing 4: Création du projet dbt

Lors de l'initialisation, choisir **bigquery** comme type de base de données.

6.3 Configuration du fichier profiles.yml

Deux fichiers profiles.yml sont nécessaires :

6.3.1 Pour les tests locaux (Windows)

Emplacement : C:\Users\AMINE\.dbt\profiles.yml

```
1 mon_projet_stg_sofac:
2   target: dev
3   outputs:
4     dev:
5       type: bigquery
6       method: service-account
7       project: pipeline-elt-test
8       dataset: dwh
9       threads: 4
10      keyfile: C:\Users\AMINE\mon_projet_stg_sofac\bigquery-key.
      json
11      location: europe-west1
```

Listing 5: profiles.yml local

6.3.2 Pour Airflow (conteneur Docker)

Emplacement : C:\Users\AMINE\mon_projet_stg_sofac\profiles.yml

```
1 mon_projet_stg_sofac :
2   target: dev
3   outputs:
4     dev:
5       type: bigquery
6       method: service-account
7       project: pipeline-elt-test
8       dataset: dwh
9       threads: 4
10      keyfile: /opt/dbt_project/bigquery-key.json
11      location: europe-west1
```

Listing 6: profiles.yml pour Airflow

[i] Information

Le chemin du keyfile est différent : sur Windows il utilise le chemin local, dans le conteneur Docker il utilise le chemin Linux monté en volume.

6.4 Structure des modèles dbt

```
1 mon_projet_stg_sofac/
2     dbt_project.yml
3     profiles.yml           (pour Airflow)
4     bigquery-key.json      (cle de service)
5     models/
6         staging/
7             sources.yml
8             stg_clients.sql
9             stg_commandes.sql
10        marts/
11            fct_ca_par_client.sql
12            schema.yml
```

Listing 7: Structure du projet dbt

6.5 Test de la connexion

```
1 cd C:\Users\AMINE\mon_projet_stg_sofac
2 dbt debug
```

Listing 8: Test de connexion dbt

Résultat attendu : All checks passed!

7 Installation d'Apache Airflow (Docker Compose)

7.1 Prérequis

1. Installer **Docker Desktop** depuis <https://www.docker.com/products/docker-desktop/>
2. Vérifier l'installation :

```
1 docker --version
2 docker compose version
```

Listing 9: Vérification Docker

7.2 Téléchargement du fichier docker-compose.yaml

```
1 # Cr er le dossier de travail
2 mkdir C:\materiels
3 cd C:\materiels
4
5 # T l charger le fichier officiel Airflow
6 curl -LfO "https://airflow.apache.org/docs/apache-airflow/2.5.1/
   docker-compose.yaml"
```

Listing 10: Téléchargement docker-compose.yaml

7.3 Modifications du docker-compose.yaml

Trois modifications à apporter :

7.3.1 1. Installer dbt-bigquery au démarrage

Trouver la ligne `_PIP_ADDITIONAL_REQUIREMENTS` et modifier :

```
1 _PIP_ADDITIONAL_REQUIREMENTS: ${_PIP_ADDITIONAL_REQUIREMENTS:-
   dbt-bigquery==1.4.3 requests}
```

Listing 11: Installation de dbt-bigquery

7.3.2 2. Monter le projet dbt en volume

Dans la section volumes de `x-airflow-common` :

```
1 volumes :
2 - ${AIRFLOW_PROJ_DIR:-.}/dags:/opt/airflow/dags
3 - ${AIRFLOW_PROJ_DIR:-.}/logs:/opt/airflow/logs
4 - ${AIRFLOW_PROJ_DIR:-.}/plugins:/opt/airflow/plugins
5 - C:/Users/AMINE/mon_projet_stg_sofac:/opt/dbt_project
```

Listing 12: Montage du projet dbt

7.3.3 3. Ajouter le nom de la machine pour NiFi

```
1 extra_hosts :  
2   - "DESKTOP-QV50A8V:host-gateway"
```

Listing 13: Configuration `extra_hosts` pour NiFi

Remplacer `DESKTOP-QV50A8V` par le nom réel de votre machine, obtenu via la commande `hostname` dans PowerShell.

7.4 Démarrage d’Airflow

```
1 cd C:\materiels  
2  
3 # Initialisation (première fois uniquement)  
4 docker compose up airflow-init  
5  
6 # Démarrage de tous les services  
7 docker compose up -d  
8  
9 # Vérification de l'état  
10 docker ps
```

Listing 14: Démarrage d’Airflow

7.5 Accès à l’interface

Ouvrir le navigateur sur : <http://localhost:8080>

Identifiants par défaut : `airflow` / `airflow`

7.6 Placement du DAG

Copier le fichier `pipeline_elt_nifi_dbt.py` dans :

```
C:\materiels\dags\pipeline_elt_nifi_dbt.py
```

8 Configuration du DAG Airflow

Le DAG orchestre le pipeline complet en 6 étapes :

1. `get_nifi_token` : Authentification sur l’API NiFi
2. `start_nifi_flow` : Démarrage du flux d’extraction
3. `wait_for_processing` : Attente de la fin du chargement
4. `stop_nifi_flow` : Arrêt du flux NiFi
5. `dbt_run` : Exécution des transformations
6. `dbt_test` : Validation de la qualité des données

8.1 Variables à configurer dans le DAG

Ouvrir le fichier `pipeline_elt_nifi_dbt.py` et modifier :

```

1 NIFI_BASE_URL = "https://DESKTOP-QV50A8V:8443/nifi-api"
2 NIFI_USERNAME = "votre_login_nifi"
3 NIFI_PASSWORD = "votre_mot_de_passe_nifi"
4
5 DBT_PROJECT_DIR = "/opt/dbt_project"
6 DBT_PROFILES_DIR = "/opt/dbt_project"
7
8 ALERT_EMAIL = "votre_email@example.com"
9 WAIT_SECONDS = 60

```

Listing 15: Variables de configuration du DAG

8.2 Planification

Le DAG est planifié pour tourner automatiquement **chaque nuit à 2h du matin** :

```

1 schedule_interval="0 2 * * *"

```

Listing 16: Planification du DAG

9 Vérification de l'installation complète

9.1 Checklist de vérification

Composant	Commande / Action	Résultat attendu
PostgreSQL	Connexion pgAdmin	Base mabase visible
NiFi	https://localhost:8443/nifi	Interface accessible
dbt	dbt debug	All checks passed!
BigQuery	Requête SQL sur staging	Tables visibles
Airflow	http://localhost:8080	Interface accessible
DAG	Trigger DAG manuel	Toutes les tâches en vert

Table 5: Checklist de vérification

9.2 Test end-to-end

[v] Vérification

Pour valider l'installation complète, déclencher le DAG manuellement dans Airflow et vérifier dans BigQuery :

```

1 SELECT * FROM 'pipeline-elt-test.dwh.fct_ca_par_client'
2 ORDER BY ca_total_livre DESC
3 LIMIT 10;

```

Si des données s'affichent, le pipeline est opérationnel de bout en bout.